From secp256k1 to ML-DSA: Designing a Quantum-Safe Multi-Admin Vault on Ethereum Sepolia

Abstract

Public blockchains are asymmetric systems: transaction histories are immutable, but signature security rests on assumptions that large-scale quantum computers are expected to erode. Ethereum EOAs rely on ECDSA over `secp256k1`, whose security is based on the elliptic-curve discrete logarithm problem (ECDLP), a problem solvable in polynomial time by Shor's algorithm on a sufficiently capable quantum computer.[1][2][3]

This work implements a hybrid mitigation path rather than a protocol rewrite. A React/MetaMask frontend, FastAPI backend, Python Dilithium2 module via liboqs, and Node/ethers.js bridge enforce post-quantum multi-admin approvals and submit real transactions to Ethereum Sepolia, with receipts verifiable on Sepolia Etherscan. The result is an operational prototype of quantum-aware treasury control: proposals are approved using lattice-based signatures, approvals are persisted, and transactions are actually mined, not merely simulated.[4][5][1]

Introduction

Ethereum's security model still leans on elliptic-curve cryptography: EOAs authenticate with ECDSA over `secp256k1`, and every published transaction reveals the corresponding public key. Under classical assumptions, recovering the private key from this public key is infeasible, but under a quantum threat model the same material becomes a high-value target embedded permanently in a public ledger.[2][3][1]

Shor's algorithm converts the ECDLP from an exponential-style brute-force problem into one solvable in polynomial time, collapsing ECC's security margin once fault-tolerant quantum hardware exists. Grover's algorithm, while less catastrophic, halves the effective security level of symmetric primitives and hash functions by providing a quadratic speedup for unstructured search. This blog describes a system that responds to these realities by re-architecting authorization: integrating lattice-based signatures into a multi-admin vault workflow while retaining Ethereum Sepolia as the settlement layer.[3][6][7][8][9][1][2]

Quantum Threat to Blockchain

The quantum threat to blockchain systems is dominated by public-key cryptography, with hashing degraded but not fundamentally broken. ECDSA over `secp256k1` assumes the hardness of computing d from $Q=dG$ in an elliptic-curve group; Shor's algorithm replaces this assumption with an efficient quantum algorithm using quantum Fourier transform–based period finding. Once such hardware is available, the private keys behind long-lived public keys can be recovered in polynomial time.[6][1][2][3]

Grover's algorithm offers a quadratic speedup for exhaustive search, reducing the effective work factor for a 256-bit hash from 2^{256} to roughly 2^{128} idealized queries. Properly sized hash functions such as Keccak remain viable, but their long-term margin is reduced. A realistic attack model is thus hybrid: classical analytics identify high-value, long-lived wallets and governance keys, while quantum resources are reserved for selectively attacking the exposed public keys behind those assets. In this context, the often-cited 2028–2035 horizon is an engineering planning window: once the cryptanalytic threshold is crossed, systems without migration paths may lose the time needed to rotate or neutralize vulnerable keys.[7][8][9][10][11][12][1]

Technical Deep Dive: ECC vs Quantum

Elliptic-curve cryptography secures Ethereum by making discrete logarithms in elliptic-curve groups computationally infeasible on classical hardware. For `secp256k1`, the private key is a scalar d and the public key $Q=dG$, with no known classical algorithm that recovers d faster than generic attacks of $\tilde{O}(\sqrt{n})$.[1][6]

Shor's algorithm embeds discrete log into a hidden subgroup problem, uses quantum Fourier transform to extract periods, and yields a polynomial-time algorithm for both factoring and discrete logarithms. For elliptic curves at blockchain key sizes, analyses indicate that a quantum computer with thousands of logical qubits could, in principle, break ECDSA. Grover's algorithm, by contrast, provides $O(\sqrt{N})$ query complexity for unstructured search, effectively halving symmetric security but not erasing it.[13][14][2][3][6][7][1]

The implication is clear: signature schemes based on discrete log or factoring face a structural break, while properly sized hash functions remain usable but with reduced margins. Any long-lived blockchain system that exposes public keys and depends on ECDSA must therefore plan for a post-quantum migration path.[12][7][1]

Why Current Blockchain Systems Are Vulnerable

Blockchains amplify cryptographic exposure because transaction and signature data are stored forever. Each Ethereum transaction reveals the ECDSA public key behind an address, turning EOAs that control treasuries, DAOs, and admin keys into durable targets for future quantum adversaries.[11][3][1]

This enables "harvest now, decrypt (or derive) later" attacks: adversaries archive today's ledger and associated signatures, then apply quantum algorithms when they become viable to extract private keys and retroactively compromise high-value assets. Governance and upgrade keys are especially exposed, as they are often static and reused over long periods, accumulating signatures and making them ideal delayed-payoff targets. Without explicit post-quantum migration plans, systems risk facing compressed, high-risk transitions under active attack.[10][15][16][12][1]

Proposed System: Quantum-Safe Multi-Admin Vault

The proposed system introduces a quantum-aware multi-admin vault that separates authorization from settlement. Authorization occurs in a post-quantum approval plane based on Dilithium-class signatures, while settlement remains on the unchanged Ethereum Sepolia network using classical ECDSA.[4][1]

The architecture has four coordinated planes:

- A React frontend with MetaMask for operator identity and network gating.
- A FastAPI backend that maintains vaults, proposals, signatures, and approvals in SQLite.
- A Python PQC module using Dilithium2 via liboqs.
- A Node/ethers.js bridge submitting transactions to Sepolia and awaiting receipts.[5][1][4]

Vault governance is threshold-based: proposals must collect a configured number of valid post-quantum signatures before the backend will construct and submit an Ethereum transaction.[1]

System Architecture and Data Flow

Frontend and Identity Layer

The frontend is a Vite/React application styled with Tailwind CSS and animated using Framer Motion. MetaMask provides the operator's session and identity, enforces the Sepolia chain ID (`0xaa36a7`), and gates access to vault creation, proposal creation, signing, approval, and execution flows. The browser does not broadcast Ethereum transactions; it uses `ethers.js` primarily for checking addresses and normalizing ETH values before calling backend APIs.[4][1]

Backend Control Plane

The FastAPI backend exposes endpoints such as `/create-vault`, `/create-proposal`, `/sign-proposal`, `/approve-proposal`, `/verify-signature`, `/execute`, and `/proposals`. State is persisted in SQLite tables (`vaults`, `vault_admins`, `proposals`, `proposal_signatures`, `approvals`), which provide an auditable record of vault configurations, signatures, approvals, and executions. PQC keypairs are stored as JSON in a `keys/` directory, either registered from external material or generated on demand.[1]

PQC Module

The `pqc/dilithium.py` module wraps the liboqs Python bindings and exposes `generate_keypair`, `sign_message`, and `verify_signature`. Keys and signatures are base64-encoded for persistence and transport, and runtime tests demonstrate successful Dilithium2 key generation, signing, and verification across the stack.[5][1]

Smart Contract and Execution Bridge

The Solidity contract `MultiAdminVault.sol` encodes threshold vault semantics on chain. It registers admins and immutable thresholds, stores proposals, prevents duplicate approvals per admin, and executes proposals only after the threshold is met, ensuring idempotent execution via a boolean guard and using low-level calls for value transfer.[1]

The execution bridge is implemented in `ethers_runner.mjs`, which creates a `JsonRpcProvider` and `Wallet` from `SEPOLIA_RPC_URL` and `SEPOLIA_PRIVATE_KEY` and then either sends direct transfers or calls `MultiAdminVault.executeProposal(...)` depending on configuration. The RPC endpoint targets a public Sepolia provider (for example, Infura), ensuring interaction with the live test network rather than a local simulator.[17][4][1]

End-to-End Data Flow

The runtime flow is:[1]

1. React + MetaMask: operator creates a vault via `/create-vault`.
2. Operator submits proposal parameters to `/create-proposal`.
3. `/sign-proposal` generates and verifies a Dilithium2 signature on a canonical JSON representation of the proposal and stores it in `proposal_signatures`.
4. `/approve-proposal` records approvals tied to verified PQC signatures and enforces per-admin uniqueness and (when configured) MetaMask-address consistency.
5. `/execute` delegates to `ethers_runner.mjs`, which submits the transaction to Sepolia, waits for `receipt.status === 1`, stores the `execution_tx_hash`, and returns it to the UI.[4][1]

The UI then renders the hash and constructs a Sepolia Etherscan link for independent verification.[1]

PQC Implementation Details

The PQC layer signs a canonical representation of each proposal rather than ad-hoc payloads. Messages include `proposal_id`, `vault_id`, `title`, `description`, `destination`, `amount_eth`, `amount_wei`, `payload`, and `created_at`, serialized as JSON with sorted keys and fixed separators to guarantee a stable byte string.[1]

The signing pipeline:[1]

1. Load and decode the admin's Dilithium private key from the `keys/` directory.
2. Use `liboqs.Signature("Dilithium2", secret_key=...)` to sign the canonical message.
3. Verify the signature with the stored public key and reject if verification fails.
4. Persist the verified signature in `proposal_signatures` keyed to the proposal and admin.[5][1]

Approvals are allowed only when a verified PQC signature already exists for that admin and proposal, and the backend can enforce that the MetaMask address used matches the one recorded during signing. This yields two independent checks: post-quantum validity of the approval material and consistency with the operator's session identity.[1]

Conceptually, this aligns with the NIST standardization trajectory: FIPS 204 defines ML-DSA, derived from CRYSTALS-Dilithium, as a primary post-quantum signature algorithm, complementing FIPS 203 (ML-KEM) and FIPS 205 (SLH-DSA, based on SPHINCS+). The prototype uses the pre-standardization "Dilithium2" API from liboqs but occupies the same ML-DSA design space.[18][19][20][12][5][1]

Real-Time Execution and Blockchain Integration

The system interacts with the live Sepolia network, not a local development chain. On execution, the backend calls `wallet.sendTransaction(...)` or `contract.executeProposal(...)`, awaits inclusion, checks `receipt.status`, and persists the `execution_tx_hash` in the proposals table.[17][4][1]

The frontend displays this hash and provides a direct Sepolia Etherscan link, enabling independent verification of gas usage, nonce behavior, and event logs. Error handling in `ethers_runner.mjs` explicitly addresses insufficient funds, gas estimation failures, contract reverts, RPC errors, and nonce desynchronization, reflecting real-world operational failure modes. The local database contains multiple proposals marked executed with Sepolia transaction hashes, confirming that the pipeline performs genuine settlement rather than mocked responses. Recent runtime data shows that this

execution path currently uses direct transfers (`contract_address = None`, `onchain_proposal_id = null`), while the contract-backed path is implemented and tested but less exercised.[4][1]

Comparison: Classical vs Post-Quantum Cryptography

Property	ECDSA (secp256k1)	Dilithium2 / ML-DSA family
Security basis	Elliptic-curve discrete log problem[1][6]	Module-LWE / Module-SIS lattice problems[1][12][20]
Quantum resistance	Broken by Shor's algorithm on large-scale quantum computers[1][2][3]	No known polynomial-time quantum break; believed secure against Shor-style attacks[1][12][20]
Public key size	~33 bytes compressed or 64 bytes uncompressed[1]	≈1.3 KB, depending on parameter set[1][12][21]
Private key size	32 bytes[1]	≈2.5–3 KB, depending on parameter set[1][12][21]
Signature size	~64–72 bytes[1]	≈2.4–2.7 KB, depending on parameter set[1][12][21]
On-chain friendliness	Highly efficient; compact artifacts and native precompiles[1]	Bandwidth- and storage-heavy; on-chain verification expensive without special support[1][12]
Verification cost	Fast and deeply integrated into Ethereum tooling[1]	Higher CPU and memory footprint in pure software implementations[1][12]
Migration posture	Legacy baseline with looming quantum obsolescence[1][12]	Standardized PQC target but operationally heavier to deploy[1][12][20]

This motivates hybrid designs like the one presented here, where PQC is introduced first at the application boundary while settlement remains classical.[12][1]

Key Insights

1. **Signatures are the primary quantum fault line.**
   For Ethereum-style systems, the immediate quantum risk is the ECDSA stack, not Keccak; Shor's algorithm directly compromises ECDLP, while Grover only compresses hash security margins.[2][3][7][1]

2. **PQC can be deployed today without modifying the EVM.**
   A service-side approval plane can enforce Dilithium-class signatures while preserving unmodified EVM execution on Ethereum Sepolia.[5][4][1]

3. **Hybrid architectures are the realistic migration path.**
   Replacing all blockchain signatures at once is impractical; separating authorization from

settlement and progressively hardening the authorization plane is a more achievable strategy.[10][12][1]

4. **Identity and authorization must be treated as distinct layers.** MetaMask provides session identity, Dilithium signatures authorize proposals, and a backend EOA signs transactions, creating layered trust boundaries that can evolve toward fully PQC-native flows.[1]

5. **Real receipts turn prototypes into engineering artifacts.** Persisted Sepolia hashes and Etherscan links provide falsifiable evidence that post-quantum approval semantics can coexist with real network settlement.[4][1]

Results and Observations

Hardhat tests confirm that `MultiAdminVault` correctly enforces threshold approvals, prevents duplicate approvals, and executes proposals once the threshold is met. The PQC module successfully generates Dilithium2 keypairs and performs end-to-end signing and verification of canonical proposal messages using liboqs.[5][1]

The backend enforces a strict "sign then approve" policy, with signatures and approvals stored separately and approvals accepted only when backed by verified PQC signatures, improving auditability and preventing accidental or malicious bypass. The SQLite database records tens of vaults and proposals, with multiple proposals marked executed and associated with Sepolia transaction hashes visible on Etherscan, confirming that the full pipeline—from post-quantum approval to blockchain settlement—is operational.[4][1]

Quantum Impact and Future Outlook

The prototype aligns with the emerging NIST PQC standards and the "harvest now, decrypt later" threat model. FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) transform PQC from candidate sets into concrete engineering targets, enabling long-term key management and authorization designs grounded in standardized algorithms.[16][19][20][18][10][12][1]

For blockchains, the urgency is higher than in many other domains: ledger data and signatures can be recorded now and attacked later, so treasuries and governance systems must begin isolating and migrating critical keys before practical quantum hardware arrives. Over the next decade, architectures that separate authorization from settlement and progressively reduce dependence on vulnerable keys will be better positioned to absorb the transition to post-quantum primitives. Future work includes moving PQC verification closer to execution (for example, on-chain ML-DSA verifiers or

protocol-level precompiles), integrating ML-KEM for key establishment, and exploring hash-based signatures such as SLH-DSA for extremely long-lived control surfaces.[15][19][11][16][18][10][12][1]

Conclusion

Quantum-safe blockchain security begins by re-architecting authorization and key management, not by immediately replacing consensus mechanisms. This multi-admin vault demonstrates a practical hybrid model: use Dilithium-class lattice signatures to secure the approval workflow, enforce threshold governance off-chain, and settle transactions on Ethereum Sepolia with verifiable receipts, all without modifying the EVM.[12][4][1]

Although the current implementation still relies on a classical EOA for final transaction submission and has not yet exercised the contract-native vault path extensively in live data, this reflects a realistic migration trajectory: deploy PQC at the application edge first, then progressively drive it deeper into the stack. As NIST's FIPS 203/204/205 standards mature and tooling around ML-DSA and related schemes improves, architectures like this can systematically reduce dependence on ECDSA, narrowing the window for "harvest now, decrypt later" attacks and preparing blockchain governance for the quantum era.[16][10][12][1]

***

1. QUANTUM_SAFE_MULTI_ADMIN_VAULT_BLOG.md (.md file of Quantum Safe Multi Admin Vault)
2. https://en.wikipedia.org/wiki/Shor's_algorithm
3. https://arxiv.org/abs/quant-ph/0301141
4. https://docs.sedge.nethermind.io/docs/networks/sepolia
5. https://openquantumsafe.org/liboqs/
6. https://www.tcs.tifr.res.in/~pgdsen/pages/writings/2007/encyclopedia/discretelog.pdf
7. https://en.wikipedia.org/wiki/Grover's_algorithm
8. https://www.quera.com/glossary/grovers-algorithm
9. https://quantum.cloud.ibm.com/learning/en/modules/computer-science/grovers
10. https://arxiv.org/pdf/2603.01091.pdf

11. https://www.federalreserve.gov/econres/feds/files/2025093pap.pdf

12. https://www.wolfssl.com/what-are-fips-203-204-and-205/

13. https://eitca.org/quantum-information/eitc-qi-qif-quantum-information-fundamentals/grovers-quantum-search-algorithm/implementing-grovers-algorithm/examination-review-implementing-grovers-algorithm/how-does-grovers-algorithm-provide-a-quadratic-speedup-compared-to-classical-search-algorithms/

14. https://www.spinquanta.com/news-detail/grovers-algorithm-how-it-speeds-up-quantum-search20250123082524

15. https://www.encryptionconsulting.com/harvest-now-decrypt-later-preparing-for-the-quantum-threat/

16. https://www.paloaltonetworks.com/cyberpedia/harvest-now-decrypt-later-hndl

17. https://www.cs.miami.edu/home/burt/learning/csc597.251/greeter_sepolia.html

18. https://csrc.nist.gov/pubs/fips/203/ipd

19. https://en.wikipedia.org/wiki/SPHINCS+

20. https://csrc.nist.gov/csrc/media/Presentations/2024/fips-204/images-media/perlner-fips-204-pqc2024.pdf

21. https://www.wolfssl.com/wolfboot-secure-boot-now-with-support-for-fips-204-ml-dsa-post-quantum-signature-algorithm/